Reviewer Pack — Matrix Rank Limitation in ACC^0 Circuits for Bounded Depth

Entry b78d3530defc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 08:05:07 UTC

Matrix Rank Limitation in ACC^0 Circuits for Bounded Depth

Entry ID: `b78d3530defc` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 08:05:07 UTC

1. Conjecture

Field A (mathematical branch): ACC_ZERO_CIRCUIT_COMPLEXITY **Field B** (complexity object): MATRIX_THEORY

Statement:

> For any ACC^0 circuit computing a matrix M of size n×n with entries in {0,1}, the rank of M over the reals is at most O((log n)^{d}), where d is the depth of the circuit. This implies that depth-d ACC^0 circuits cannot approximate high-rank matrices beyond this bound.

Rationale (proposer's reasoning):

> This conjecture leverages linear algebra to establish a structural limitation on ACC^0 circuits. By bounding the real rank of matrices computable by shallow ACC^0 circuits, it provides an alternative pathway to NEXP vs ACC^0 separation, avoiding Fourier-analytic and tropical methods used in prior work.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8be6365aea94146c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    depth = 5

    # Generate a random binary matrix M of size n×n
    M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    # Compute the real rank of M over the reals
    def gaussian_elimination(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = min(m, n)
        for i in range(rank):
            if matrix[i][i] == 0:
                for j in range(i + 1, m):
                    if matrix[j][i] != 0:
                        matrix[i], matrix[j] = matrix[j], matrix[i]
                        break
                else:
                    rank -= 1
                    continue
            for j in range(m):
                if j != i:
                    factor = Fraction(matrix[j][i], matrix[i][i])
                    for k in range(n):
                        matrix[j][k] -= factor * matrix[i][k]
        return rank

    real_rank = gaussian_elimination(M)

    # Simulate ACC^0 circuits of varying depths to approximate the matrix
    def simulate_acc0_circuit(matrix, depth):
        m, n = len(matrix), len(matrix[0])
        if depth == 1:
            return sum(sum(row) for row in matrix)
```

```
        else:
            half_matrix = [row[:n//2] + row[n//2:] for row in matrix]
            left_rank = simulate_acc0_circuit(half_matrix, depth-1)
            right_rank = simulate_acc0_circuit(half_matrix, depth-1)
            return max(left_rank, right_rank)

    acc0_rank_bound = simulate_acc0_circuit(M, depth)

    # Validate if the rank bound scales as O((log n)^d) for depth d
    expected_rank_bound = Fraction((math.log2(n) ** depth), 1).limit_denominator()

    return {
        "metric_name": "Rank Bound",
        "metric_value": acc0_rank_bound,
        "instances_tested": 1,
        "conjecture_holds": acc0_rank_bound <= expected_rank_bound,
        "counterexample": "" if acc0_rank_bound <= expected_rank_bound else f"Depth {depth} ACC^0
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ccdfa34.py", line 79, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ccdfa34.py", line 63, in run_trial
    expected_rank_bound = Fraction((math.log2(n) ** depth), 1).limit_denominator()
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to TypeError in Fraction initialization, preventing data collection. The crash indicates a code error rather than a mathematical refutation. | next: Fix the Fraction argument type by converting log2(n)^depth to a rational number representation

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	78472
2	propose	ollama_remote	qwen3:8b	0	0	110924
3	preregistration	ollama_remote	qwen3:8b	0	0	27447
4	novelty	ollama_remote	qwen3:8b	0	0	24237
5	novelty	ollama_remote	qwen3:8b	0	0	20860
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14430
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7688
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9889
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9880
10	judge	ollama_remote	qwen3:8b	0	0	21297

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 325123 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b78d3530defc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b78d3530defc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b78d3530defc.tar.gz` (if generated)